NOTE 07 - HOW DNS RESOLUTION WORKS, STEP BY STEP

DNS (Domain Name System) is the internet's naming service. Humans use names such as `www.example.com`; networks route to IP addresses. DNS is the lookup that converts one into the other, and it is a distributed, hierarchical, cached database - no single server knows every name.

The cast of servers

Server	Role
Stub resolver	The small client inside the operating system / browser that asks the question.
Recursive resolver	Usually provided by the ISP, or a public one (1.1.1.1, 8.8.8.8). It does all the chasing on the client's behalf and caches the answer.
Root server	13 logical root server clusters. It does not know `example.com`; it points to the right TLD server.
TLD server	Handles a top-level domain (`.com`, `.org`, `.uk`). It points to the domain's authoritative servers.
Authoritative nameserver	Holds the real records for the domain. This is where the answer finally comes from.

The resolution walk-through

Step 0 - the user types the name. `https://shop.example.com/checkout` is entered in the browser.

Step 1 - local caches are checked, in this order.

1. Browser cache - the browser keeps recently resolved names for a short time.
2. Operating-system cache and the local hosts file (`C:\Windows\System32\drivers\etc\hosts` or `/etc/hosts`) - a manual override that wins over DNS.
3. The stub resolver's cache.

If any of them has a valid, unexpired entry, resolution stops here and no packet leaves the machine. This is the fastest path and it is what happens for most requests to popular sites.

Step 2 - the query goes to the recursive resolver. The stub resolver sends a recursive query (usually UDP port 53) to the resolver (for example `1.1.1.1`): "please give me the final answer for `shop.example.com`." Because the client asked for recursion, the resolver now does all the work.

Step 3 - the resolver checks its own cache. The recursive resolver serves thousands of users, so its cache is large and hits often. If there is a valid cached answer, it returns it immediately and the process ends. Only a cache miss continues.

Step 4 - root server. The resolver asks a root server: "where do I find `.com`?" The root server does not answer the question itself; it returns a referral - the addresses of the `.com` TLD servers. (This is an iterative query: each server returns the next place to ask.)

Step 5 - TLD server. The resolver asks a `.com` TLD server: "where do I find `example.com`?" Again, no final answer - it returns a referral with the addresses of the authoritative nameservers for `example.com`, such as `ns1.example.net`.

Step 6 - authoritative nameserver. The resolver finally asks the authoritative server for `shop.example.com`. Because this server owns the zone, it returns the authoritative answer - for example an A record with IPv4 address `203.0.113.45` (or a AAAA record for IPv6, or a CNAME that starts a second lookup).

Step 7 - the answer travels back and is cached. The resolver sends the answer back to the stub resolver, which hands it to the browser. Every cache along the way stores the record for the number of seconds given by its TTL (Time To Live) field. The browser then opens a TCP connection to port 443 (for HTTPS) on that IP address.

The same flow as a diagram

```
Browser  ->  Stub resolver  ->  [ local caches ]  ->  Recursive resolver (ISP / 1.1.1.1)
                                                                  |
                                                                  +--> cache hit? -> answer
                                                                  |
                                                                  +--> Root server        -> "ask .com TLD
"
                                                                  +--> .com TLD server    -> "ask example.
com NS"
                                                                  +--> Authoritative NS   -> A 203.0.113.4
5 (TTL)
                                                                  |
     Browser <-- answer (cached for TTL seconds) <------------+
```

Important record types

Record	Meaning
A	Name -> IPv4 address
AAAA	Name -> IPv6 address
CNAME	Name -> another name (an alias; the resolver then resolves the target)
MX	The mail servers for the domain, with a priority value
TXT	Free text - SPF/DKIM/DMARC, domain verification, and a favourite of DNS tunnelling
NS	The authoritative nameservers for the zone
PTR	Reverse lookup: IP -> name
SOA	Start of authority - zone metadata and timers

Why a SOC analyst cares

- DNS is a goldmine of evidence. Very few organisations encrypt DNS, so DNS logs show which domains every host contacted, and when.
- Common abuse: DNS tunnelling (T1071.004 / T1048.001) hides command-and-control or stolen data in query names or TXT records; DGAs generate thousands of random-looking names; long, high-entropy subdomains and very high query rates are the usual red flags.
- Caching affects investigation: the TTL explains why an old malicious answer may still be in a client's cache, and why two hosts can see different IPs for the same name.
- Practical commands used during triage: `nslookup <name>` and `dig <name>` (Linux) to see the records, `dig +trace` to watch the delegation chain, and `nslookup -type=TXT <name>` to inspect TXT records for exfiltration clues.

Key points to remember

- Resolution is hierarchical (root -> TLD -> authoritative) and cached at every level.
- The recursive resolver does the chasing; every question after that is an iterative referral.
- The authoritative nameserver is the only server that gives the definitive answer.
- TTL decides how long that answer stays cached, at every layer.

- The final IP address is what the browser then connects to - usually on port 443.